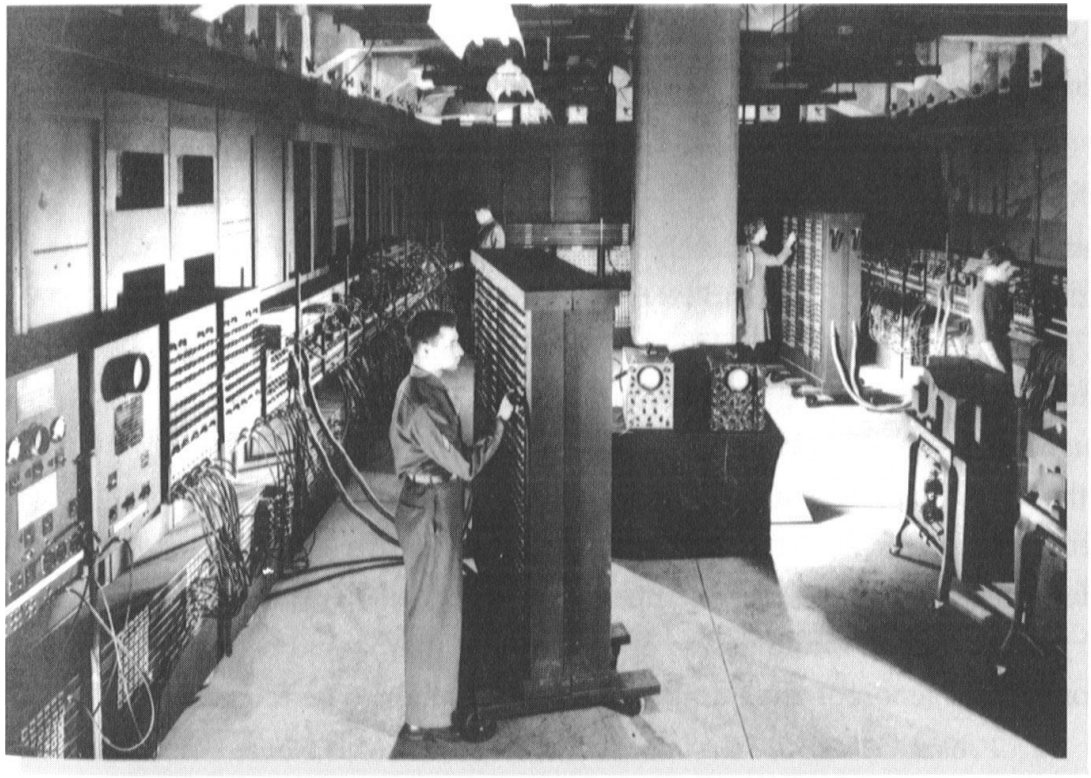


指令系统（THINPAD）

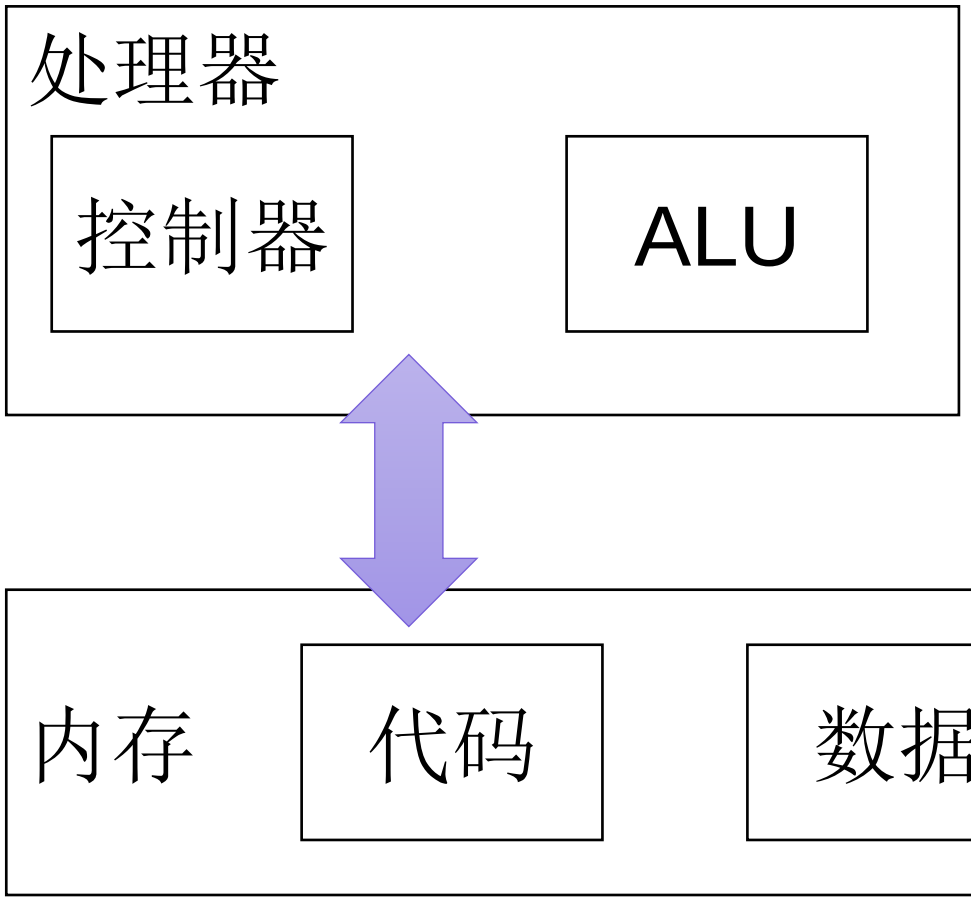
2026年秋

计算机基本组成



- 计算机理论基础诞生超过60年
  - Turing
  - Shannon
  - Von Neumann
- 组成计算机的关键部件也没有大的改变
  - CPU
    - Data Path
    - Controller
  - Memory
  - IO

Von Neumann 计算机



冯诺依曼结构
（普林斯顿结构）
哈佛结构

存储程序、二进制、体系结构

内容概要

- 计算机程序与指令系统
- RISC-V指令系统简介
- THINPAD指令系统

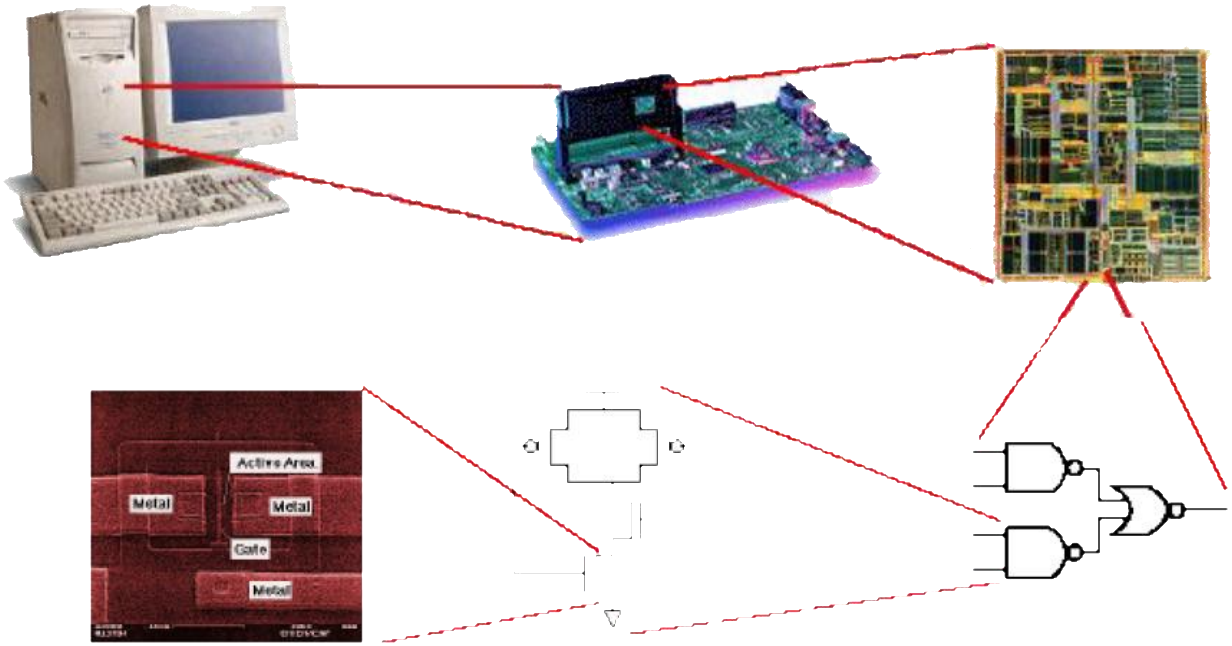
计算机程序

- **Computer programs**(also **software programs**, or just **programs**) are instructions for a computer. A computer requires programs to function, and a computer program does nothing unless its instructions are executed by a central processor. Computer programs are either executable programs or the source code from which executable programs are derived (e.g., compiled).

- 程序员和计算机硬件之间交互的语言
  - （自然语言）
  - 高级语言
  - 汇编语言
  - 机器语言

计算机剖析

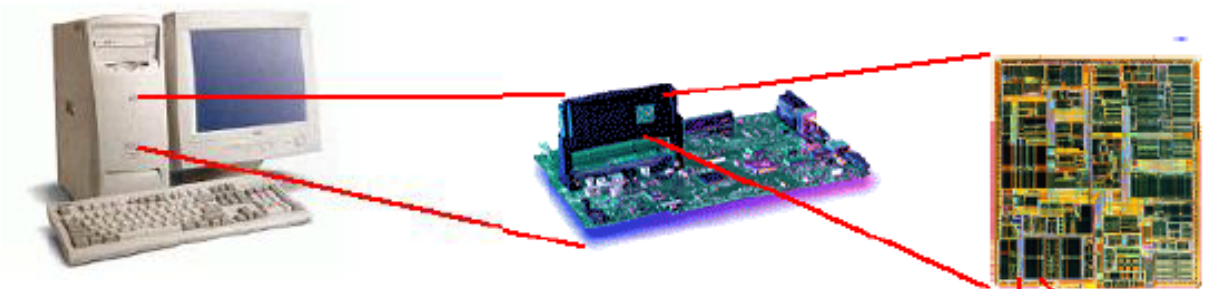
计算机系统　　计算机结构　　计算机组成与实现



生产制造　　电路设计　　逻辑设计

计算机语言与计算机层次

计算机系统　　计算机结构　　计算机组成和实现



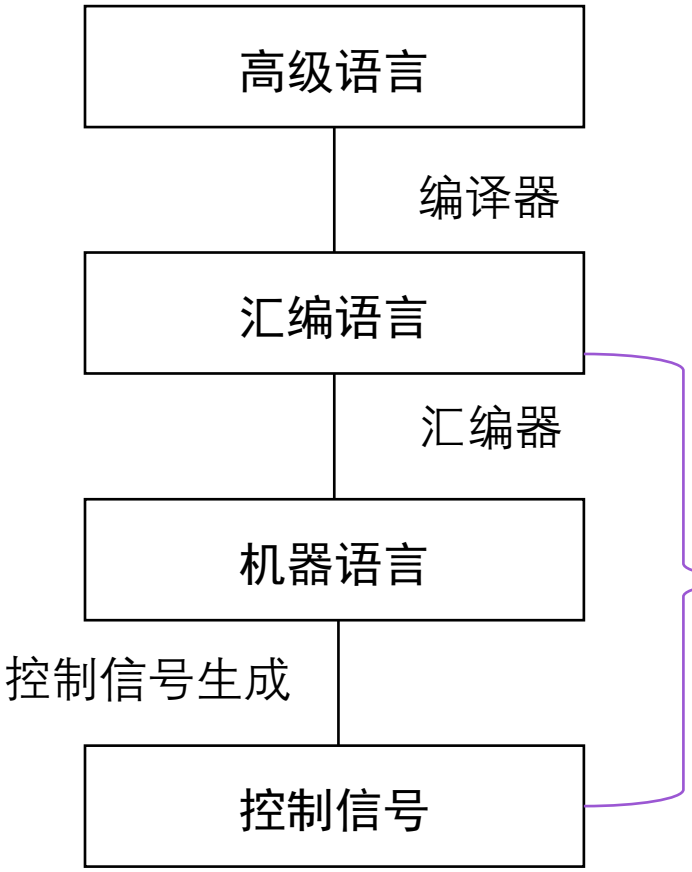
```
while (event = getnext()) {
     /* process event */
     switch (event->type) {
       case BUTTONUP:
            win = event->W;
            if (!win) break;
            do_button (win);
            break;
       case BUTTONDOWN:
            ...
     }
...
```

```
jal _getnext
ori $a0,$0,0
lw $t0,8($v0)
lw $t0,12($t0)
beq $t0,0,0x401834
li $t1,4
beq $t0,$t1,0x4018a0
```

```
0x0c004841
0x00000000
0x34040000
0x8c480008
0x00000000
0x8d08000c
0x10001834
0x00000000
0x24090004
0x11090002
...
```

高级语言　　汇编语言　　机器语言

计算机的层次结构




```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw t0, t2, 0
lw t1, t2, 4
sw t1, t2, 0
sw t0, t2, 4
```

计算机组成

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```

程序举例(sum.s)

```
int sum()
{
   int sum = 0;
   for (int i = 1; i < 11; i++)
      sum += i;
   return sum
}
```

-Og版本

```
sum:
    li   a5,1
    li   a0,0
.L2:
    li   a4,10
    bgt  a5,a4,.L4
    add  a0,a0,a5
    addi a5,a5,1
    j    .L2
.L4:
    ret
```

-O2版本

```
sum:
    li  a0,55
    ret
```

程序举例(fib.s)

```
int fibo[10];
void fib()
{
   int i;
   fibo[0]=1; fibo[1] =1;
   for ( i=2 ; i<10; i++)
      fibo[i]= fibo[i-1] + fibo[i-2];
}
```

```
    .align  2
    .globl  fib
    .type   fib, @function
fib:
    lui   a5,%hi(fibo)          # high value of fibo in a5
    addi  a4,a5,%lo(fibo)  # set a4 = address of fibo
    li    a3,1                  # a3 = 1
    addi  a5,a5,%lo(fibo)  # set a5 = address of fibo
    sw    a3,0(a4)              # fibo[0] = 1
    sw    a3,4(a4)              # fibo[1] = 1
    addi  a2,a5,32          # a2 = address of fibo[8]
    li    a4,1                  # a4 = 1
    j     .L3                   # goto L3
.L5:
    lw    a4,4(a5)              # a4 = a5[1]
    lw    a3,0(a5)              # a3 = a5[0]
.L3:
    add   a4,a4,a3              # a4 = a4 + a3
    sw    a4,8(a5)              # a5[2] = a4
    addi a5,a5,4          # a5 += 4 (points to the next en
try)
    bne   a5,a2,.L5          # not finished? goto L5
    ret
```

高级语言、汇编语言、机器语言

- 高级语言，又称算法语言，着重面向解决实际问题所用的算法，瞄准的是如何使程序设计人员能够方便地写出处理问题和解题过程的程序，力争使程序设计工作的效率更高。
- 汇编语言是对计算机机器语言进行符号化处理的结果,再增加一些为方便程序设计而实现的扩展功能。
- 机器语言是计算机硬件能直接识别和运行的指令的集合,是二进制码组成的指令

冯诺依曼结构计算机

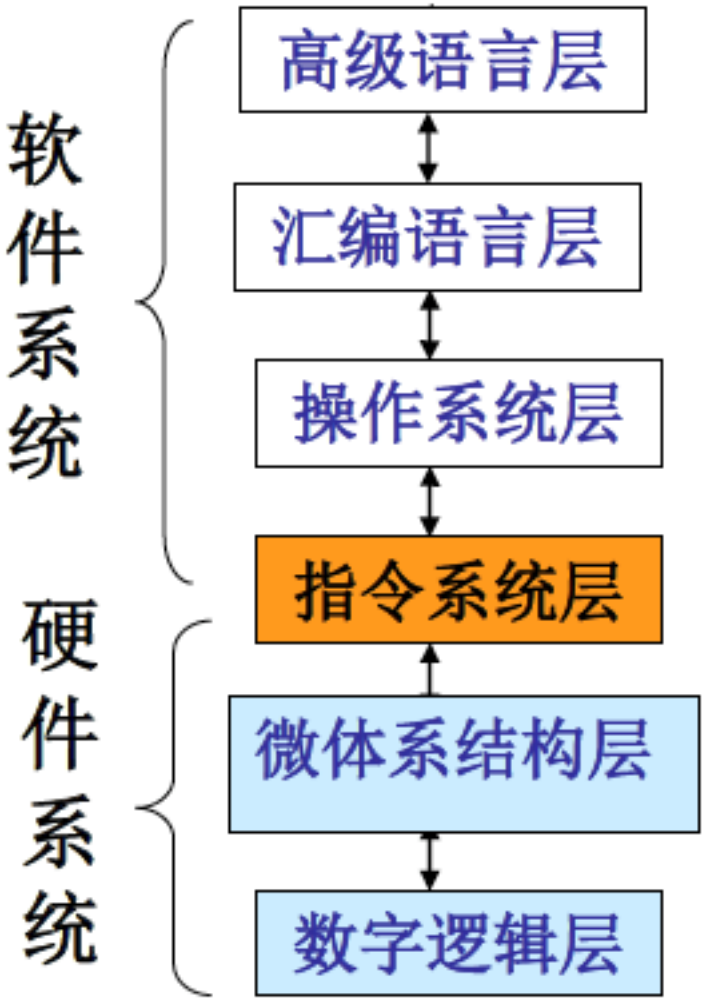
- 存储程序计算机
  - 程序由指令构成
  - 程序功能通过指令序列描述
  - 指令序列在存储器中顺序存放
- 顺序执行指令
  - PC指向需要执行的指令
  - 从存储器中读出指令执行
  - 读取完成之后，PC自增，指向下一条指令

指令和指令系统

- 计算机系统由硬件和软件两大部分组成。硬件指由中央处理器、存储器以及外围设备等组成的实际装置。软件是为了使用计算机而编写的各种系统的和用户的程序，程序由一个序列的计算机指令组成。
- 指令是计算机运行的最小的功能单元，是指挥计算机硬件运行的命令，是由多个二进制位组成的位串，是计算机硬件可以直接识别和执行的信息体。指令中应指明指令所完成的操作，并明确操作对象。
- 一台计算机提供的全部指令构成该计算机的指令系统。指令用于程序设计人员告知计算机执行一个最基本运算、处理功能，多条指令可以组成一个程序，完成一项预期的任务。

指令系统地位

- 可以从6 个层次分析和看待计算机系统的基本组成。
- 指令系统层处在硬件系统和软件系统之间，是硬、软件之间的接口部分，对两部分都有重要影响。
- 硬件系统用于实现每条指令的功能，解决指令之间的衔接关系；
- 软件由按一定规则组织起来的许多条指令组成，完成一定的数据运算或者事务处理功能。



计算机系统的层次结构

指令的功能分类

- 数据运算指令
  - 算数运算，逻辑运算
- 数据传输指令
  - 内存/寄存器，寄存器/寄存器
- 控制指令
  - 无条件跳转，条件跳转，子程序的支持（调用和返回）
- 输入输出指令
  - 与输入输出端口的数据传输（输入输出模型如何）
- 其它指令
  - 停机、开/关中断、空操作、特权指令、设置条件码

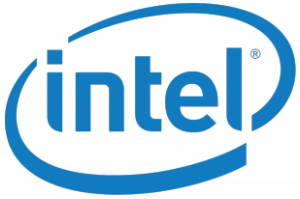
指令格式

- 指令格式：操作码，操作数地址的二进制分配方案



  - 操作码：指令的操作功能
  - 操作数地址：操作数存放的地址，或者操作数本身
- 指令字：完整的一条指令的二进制表示
- 指令字长：指令字中二进制代码的位数
  - 机器字长：计算机能够直接处理的二进制数据的位数
  - 指令字长（字节倍数）：0.5，1，2，……个机器字长
  - 定长指令字结构 变长指令字结构
  - 定长操作码 扩展操作码

不同的指令集体系结构



x86

Designer	Intel, AMD
Bits	16-bit, 32-bit and 64-bit
Introduced	1978 (16-bit), 1985 (32-bit), 2003 (64-bit)
Design	CISC
Type	Register-memory
Encoding	Variable (1 to 15 bytes)
Endianness	Little

笔记本电脑，台式机，服务器
(Core i3, i5, i7, M)
x86 Instruction Set



ARM architectures

Designer	ARM Holdings
Bits	32-bit, 64-bit
Introduced	1985;
Design	RISC
Type	Register-Register
Encoding	AArch64/A64 and AArch32/A32 use 32-bit instructions, T32 (Thumb-2) uses mixed 16- and 32-bit instructions. ARMv7 user-space compatibility[1]
Endianness	Bi (little as default)

智能手机
(iPhone, iPad, Raspberry Pi)
ARM Instruction Set



MIPS

Designer	MIPS Technologies, Inc.
Bits	64-bit (32→64)
Introduced	1981;
Design	RISC
Type	Register-Register
Encoding	Fixed
Endianness	Bi

Digital home & networking equipment
(Blu-ray, PlayStation 2)
MIPS Instruction Set

复杂指令集（CISC）/精简指令集（RISC）

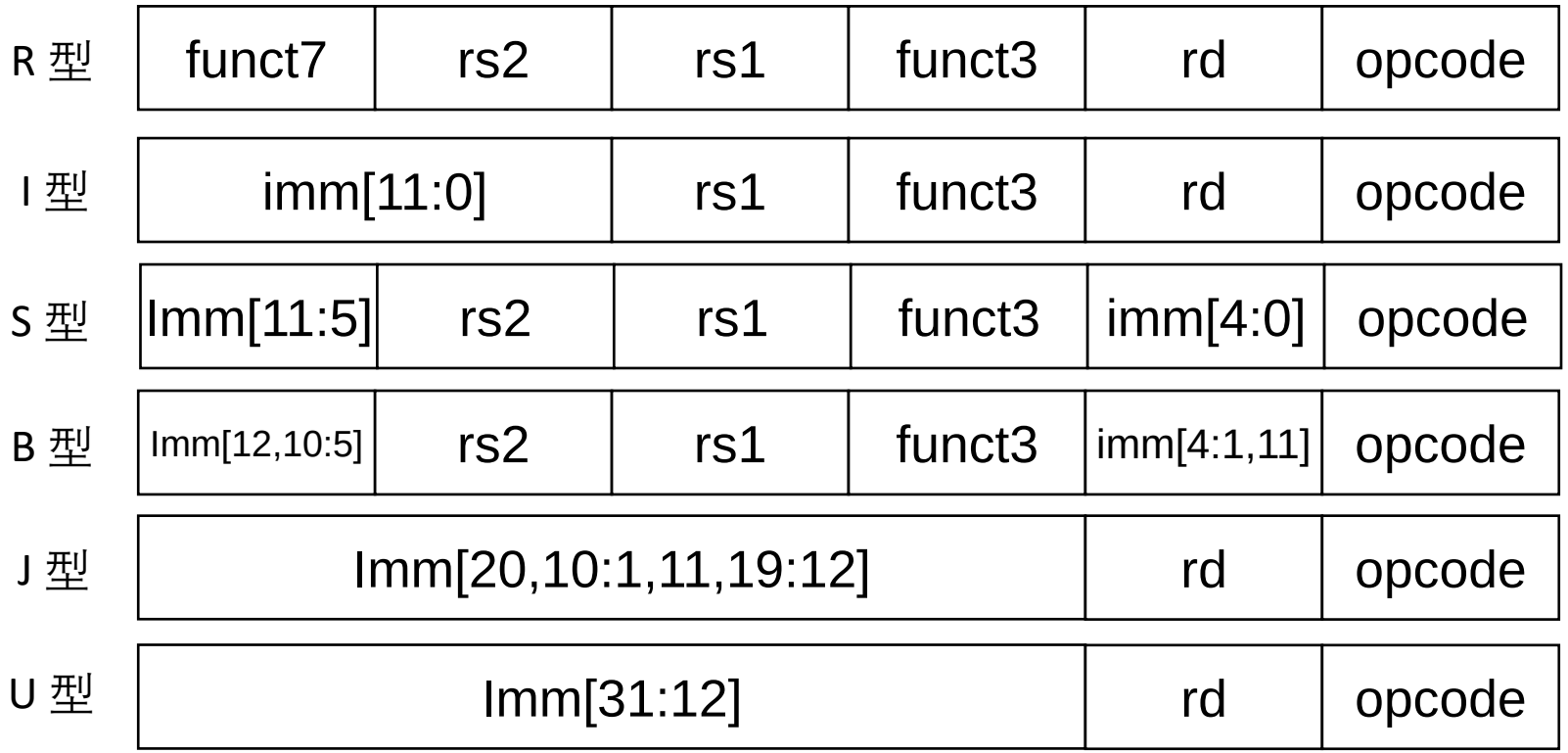
- CISC（Complex Instruction Set Computing）
  - 早期的趋势：为了能够做复杂的操作，增加越来越多的指令
  - 导致学习和理解上的困难
  - 导致硬件复杂（可能会变慢）
- RISC（Reduced Instruction Set Computing ）
  - 使用更简单，更小的指令集，会更加容易获得快速的硬件
  - 如果需要做复杂操作的话，让软件来做，组合几条简单的指令来完成工作

RISC-V指令集历史

- 加州大学伯克利分校Krste Asanovic教授、Andrew Waterman和Yunsup Lee等开发人员于2010年发明。
  - 其中"RISC"表示精简指令集，而其中"V"表示伯克利分校从RISC I开始设计的第五代指令集。
- 基于BSD协议许可的免费开放的指令集架构
- 适合多层次计算机系统
  - 从 微控制器 到 超级计算机
  - 支持大量定制与加速功能
  - 32bit, 64bit, 128bit
- 规范由RISC-V非营利性基金会维护
  - RISC-V基金会负责维护RSIC-V指令集标准手册与架构文档

规整的指令编码

- 所有通用寄存器在指令码的位置是一样的，方便译码阶段的使用
- 所有的指令都是32位字长，有 6 种指令格式：寄存器型，立即数型，存储型，分支指令、跳转指令和大立即数



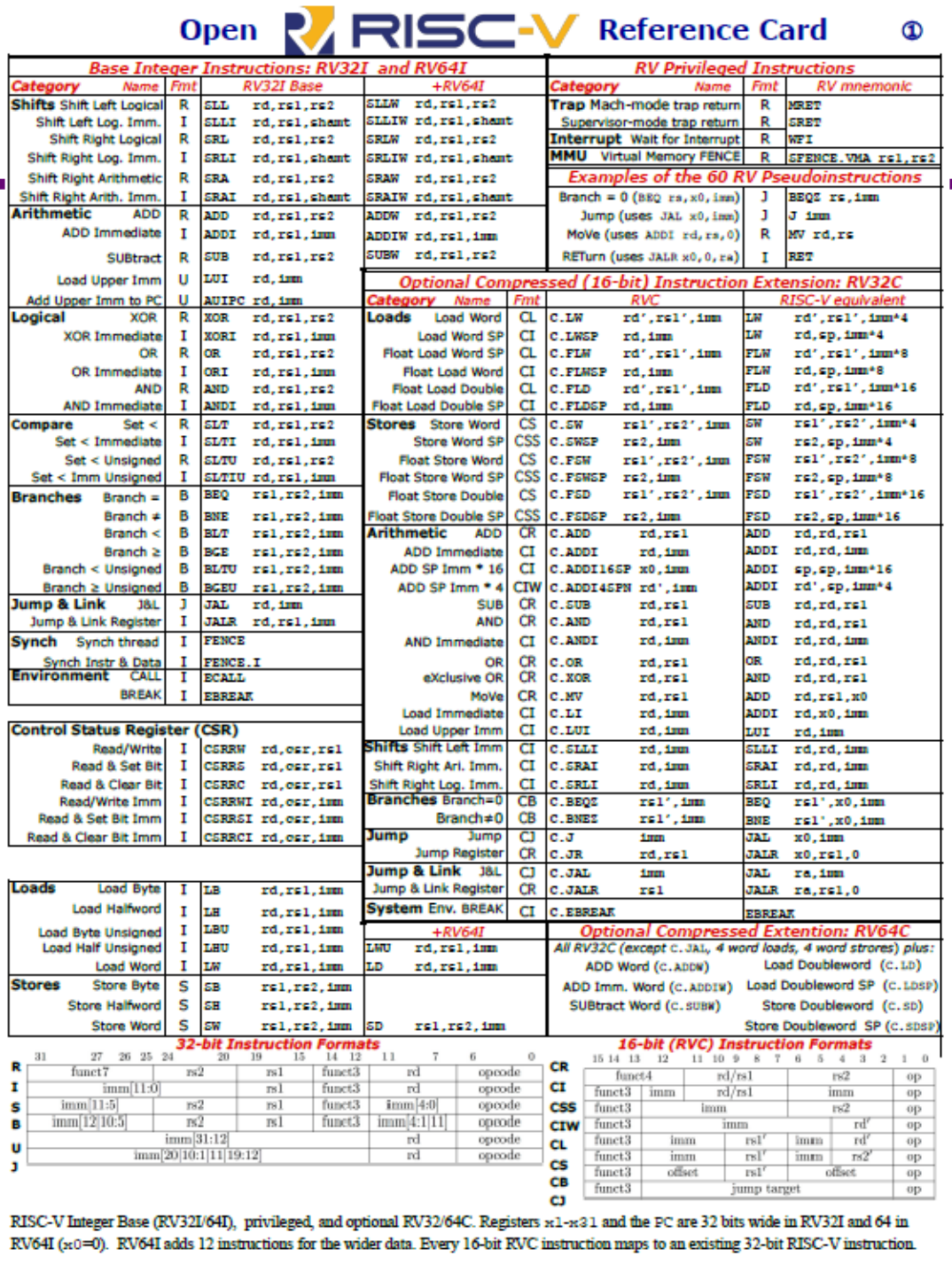
RISC-V 的指令集

- RISC-V官方指令集手册

https://docs.riscv.org/reference/isa/v20260120/index.html

- 中文简化版

http://riscvbook.com/chinese/RISC-V-Reader-Chinese-v2p1.pdf



Open RISC-V Reference Card ①

Base Integer Instructions: RV32I and RV64I

Category	Name	Fmt	RV32I Base	+RV64I
Shifts	Shift Left Logical	R	SLL rd,rs1,rs2	SLLW rd,rs1,rs2
	Shift Left Log. Imm.	I	SLLI rd,rs1,shamt	SLLIW rd,rs1,shamt
	Shift Right Logical	R	SRL rd,rs1,rs2	SRLW rd,rs1,rs2
	Shift Right Log. Imm.	I	SRLI rd,rs1,shamt	SRLIW rd,rs1,shamt
	Shift Right Arithmetic	R	SRA rd,rs1,rs2	SRAW rd,rs1,rs2
	Shift Right Arith. Imm.	I	SRAI rd,rs1,shamt	SRAIW rd,rs1,shamt
Arithmetic	ADD	R	ADD rd,rs1,rs2	ADDW rd,rs1,rs2
	ADD Immediate	I	ADDI rd,rs1,imm	ADDIW rd,rs1,imm
	SUBtract	R	SUB rd,rs1,rs2	SUBW rd,rs1,rs2
	Load Upper Imm	U	LUI rd,imm	
	Add Upper Imm to PC	U	AUIPC rd,imm	
Logical	XOR	R	XOR rd,rs1,rs2	
	XOR Immediate	I	XORI rd,rs1,imm	
	OR	R	OR rd,rs1,rs2	
	OR Immediate	I	ORI rd,rs1,imm	
	AND	R	AND rd,rs1,rs2	
	AND Immediate	I	ANDI rd,rs1,imm	
Compare	Set <	R	SLT rd,rs1,rs2	
	Set < Immediate	I	SLTI rd,rs1,imm	
	Set < Unsigned	R	SLTU rd,rs1,rs2	
	Set < Imm Unsigned	I	SLTIU rd,rs1,imm	
Branches	Branch =	B	BEQ rs1,rs2,imm	
	Branch ≠	B	BNE rs1,rs2,imm	
	Branch <	B	BLT rs1,rs2,imm	
	Branch ≥	B	BGE rs1,rs2,imm	
	Branch < Unsigned	B	BLTU rs1,rs2,imm	
	Branch ≥ Unsigned	B	BGEU rs1,rs2,imm	
Jump & Link	J&L	J	JAL rd,imm	
	Jump & Link Register	I	JALR rd,rs1,imm	
Synch	Synch thread	I	FENCE	
	Synch Instr & Data	I	FENCE.I	
Environment	CALL	I	ECALL	
	BREAK	I	EBREAK	

Control Status Register (CSR)

Read/Write	I	CSRRW	rd,csr,rs1
Read & Set Bit	I	CSRRS	rd,csr,rs1
Read & Clear Bit	I	CSRRC	rd,csr,rs1
Read/Write Imm	I	CSRRWI	rd,csr,imm
Read & Set Bit Imm	I	CSRRSI	rd,csr,imm
Read & Clear Bit Imm	I	CSRRCI	rd,csr,imm

Loads	Load Byte	I	LB	rd,rs1,imm
	Load Halfword	I	LH	rd,rs1,imm
	Load Byte Unsigned	I	LBU	rd,rs1,imm
	Load Half Unsigned	I	LHU	rd,rs1,imm
	Load Word	I	LW	rd,rs1,imm
Stores	Store Byte	S	SB	rs1,rs2,imm
	Store Halfword	S	SH	rs1,rs2,imm
	Store Word	S	SW	rs1,rs2,imm

RV Privileged Instructions

Category	Name	Fmt	RV mnemonic
Trap	Mach-mode trap return	R	MRET
	Supervisor-mode trap return	R	SRET
Interrupt	Wait for Interrupt	R	WFI
MMU	Virtual Memory FENCE	R	SFENCE.VMA rs1,rs2

Examples of the 60 RV Pseudoinstructions

Branch = 0 (BEQ rs,x0,imm)	J	BEQZ rs,imm
Jump (uses JAL x0,imm)	J	J imm
MoVe (uses ADDI rd,rs,0)	R	MV rd,rs
RETurn (uses JALR x0,0,ra)	I	RET

Optional Compressed (16-bit) Instruction Extension: RV32C

Category	Name	Fmt	RVC	RISC-V equivalent
Loads	Load Word	CL	C.LW rd',rs1',imm	LW rd',rs1',imm*4
	Load Word SP	CI	C.LWSP rd,imm	LW rd,sp,imm*4
	Float Load Word SP	CL	C.FLW rd',rs1',imm	FLW rd',rs1',imm*8
	Float Load Word	CI	C.FLWSP rd,imm	FLW rd,sp,imm*8
	Float Load Double	CL	C.FLD rd',rs1',imm	FLD rd',rs1',imm*16
	Float Load Double SP	CI	C.FLDSP rd,imm	FLD rd,sp,imm*16
Stores	Store Word	CS	C.SW rs1',rs2',imm	SW rs1',rs2',imm*4
	Store Word SP	CSS	C.SWSP rs2,imm	SW rs2,sp,imm*4
	Float Store Word	CS	C.FSW rs1',rs2',imm	FSW rs1',rs2',imm*8
	Float Store Word SP	CSS	C.FSWSP rs2,imm	FSW rs2,sp,imm*8
	Float Store Double	CS	C.FSD rs1',rs2',imm	FSD rs1',rs2',imm*16
	Float Store Double SP	CSS	C.FSDSP rs2,imm	FSD rs2,sp,imm*16
Arithmetic	ADD	CR	C.ADD rd,rs1	ADD rd,rd,rs1
	ADD Immediate	CI	C.ADDI rd,imm	ADDI rd,rd,imm
	ADD SP Imm * 16	CI	C.ADDI16SP x0,imm	ADDI sp,sp,imm*16
	ADD SP Imm * 4	CIW	C.ADDI4SPN rd',imm	ADDI rd',sp,imm*4
	SUB	CR	C.SUB rd,rs1	SUB rd,rd,rs1
	AND	CR	C.AND rd,rs1	AND rd,rd,rs1
	AND Immediate	CI	C.ANDI rd,imm	ANDI rd,rd,imm
	OR	CR	C.OR rd,rs1	OR rd,rd,rs1
	eXclusive OR	CR	C.XOR rd,rs1	AND rd,rd,rs1
	MoVe	CR	C.MV rd,rs1	ADD rd,rs1,x0
	Load Immediate	CI	C.LI rd,imm	ADDI rd,x0,imm
	Load Upper Imm	CI	C.LUI rd,imm	LUI rd,imm
Shifts	Shift Left Imm	CI	C.SLLI rd,imm	SLLI rd,rd,imm
	Shift Right Ari. Imm.	CI	C.SRAI rd,imm	SRAI rd,rd,imm
	Shift Right Log. Imm.	CI	C.SRLI rd,imm	SRLI rd,rd,imm
Branches	Branch=0	CB	C.BEQZ rs1',imm	BEQ rs1',x0,imm
	Branch≠0	CB	C.BNEZ rs1',imm	BNE rs1',x0,imm
Jump	Jump	CJ	C.J imm	JAL x0,imm
	Jump Register	CR	C.JR rd,rs1	JALR x0,rs1,0
Jump & Link	J&L	CJ	C.JAL imm	JAL ra,imm
	Jump & Link Register	CR	C.JALR rs1	JALR ra,rs1,0
System	Env. BREAK	CI	C.EBREAK	EBREAK

+RV64I

LWU	rd,rs1,imm		
LD	rd,rs1,imm		
SD	rs1,rs2,imm		

Optional Compressed Extention: RV64C

All RV32C (except C.JAL, 4 word loads, 4 word stores) plus:

- ADD Word (C.ADDW)
- ADD Imm. Word (C.ADDIW)
- SUBtract Word (C.SUBW)
- Load Doubleword (C.LD)
- Load Doubleword SP (C.LDSP)
- Store Doubleword (C.SD)
- Store Doubleword SP (C.SDSP)

32-bit Instruction Formats

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
J	imm[20|10:1|11|19:12]					rd	opcode

16-bit (RVC) Instruction Formats

	15 14 13 12	11 10	9 8 7	6 5 4 3 2	1 0
CR	funct4	rd/rs1		rs2	op
CI	funct3 | imm	rd/rs1		imm	op
CSS	funct3	imm		rs2	op
CIW	funct3	imm		rd'	op
CL	funct3	imm	rs1'	imm | rd'	op
CS	funct3	imm	rs1'	imm | rs2'	op
CB	funct3	offset	rs1'	offset	op
CJ	funct3	jump target			op

RISC-V Integer Base (RV32I/64I), privileged, and optional RV32/64C. Registers x1-x31 and the PC are 32 bits wide in RV32I and 64 in RV64I (x0=0). RV64I adds 12 instructions for the wider data. Every 16-bit RVC instruction maps to an existing 32-bit RISC-V instruction.

RISC-V 指令系统

- RV32I - 基础整数指令、
- 特权指令、
- RV32F – 单精度浮点指令、
- RV32D – 双精度浮点指令、
- 扩展指令

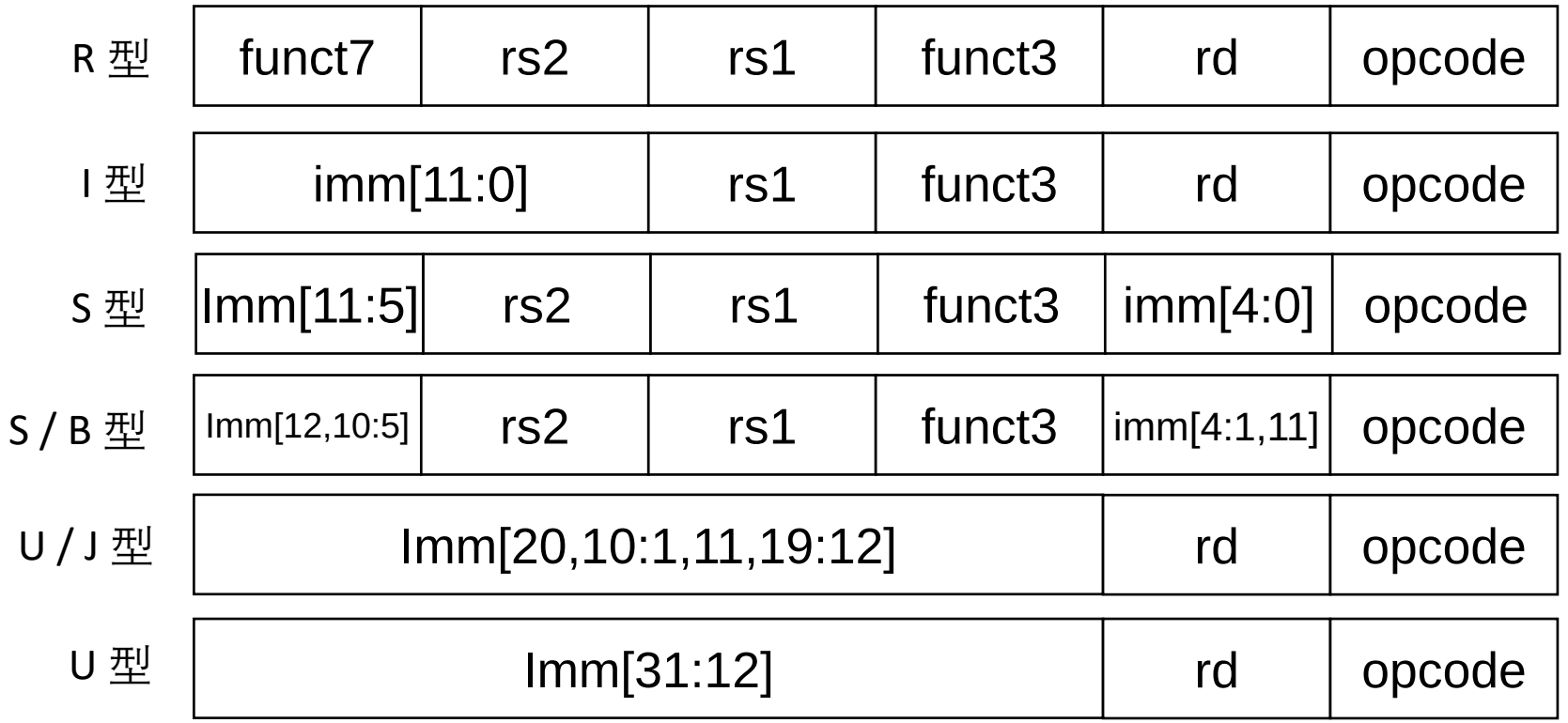
- RV32
  - 面向32位处理器的指令系统
- RV64
  - 64位字长的RISC-V指令集

ThinPAD RISC-V指令系统

- 采用与RV32I兼容的指令格式
  - 32位固定字长
  - 操作码位置及长度固定
  - 寻址方式简单
- 供设计有19+3+6+1条指令
  - 19条是基础指令，可以用于执行基本的监控程序（不包括异常与中断，不包括虚拟地址）
  - 3条是运行RV64监控程序需要的指令
  - 6条是支持优先级，支持中断与异常的指令
  - 1条是支持虚拟地址的指令（没有TLB的话可实现为Nop）
  - 可根据需要进行扩展
- 作为本课程教学实验的指令系统

RISC-V 指令格式（32位）

- 所有的指令都是32位字长，有 6 种指令格式：寄存器型，立即数型，存储型，分支指令、跳转指令和大立即数

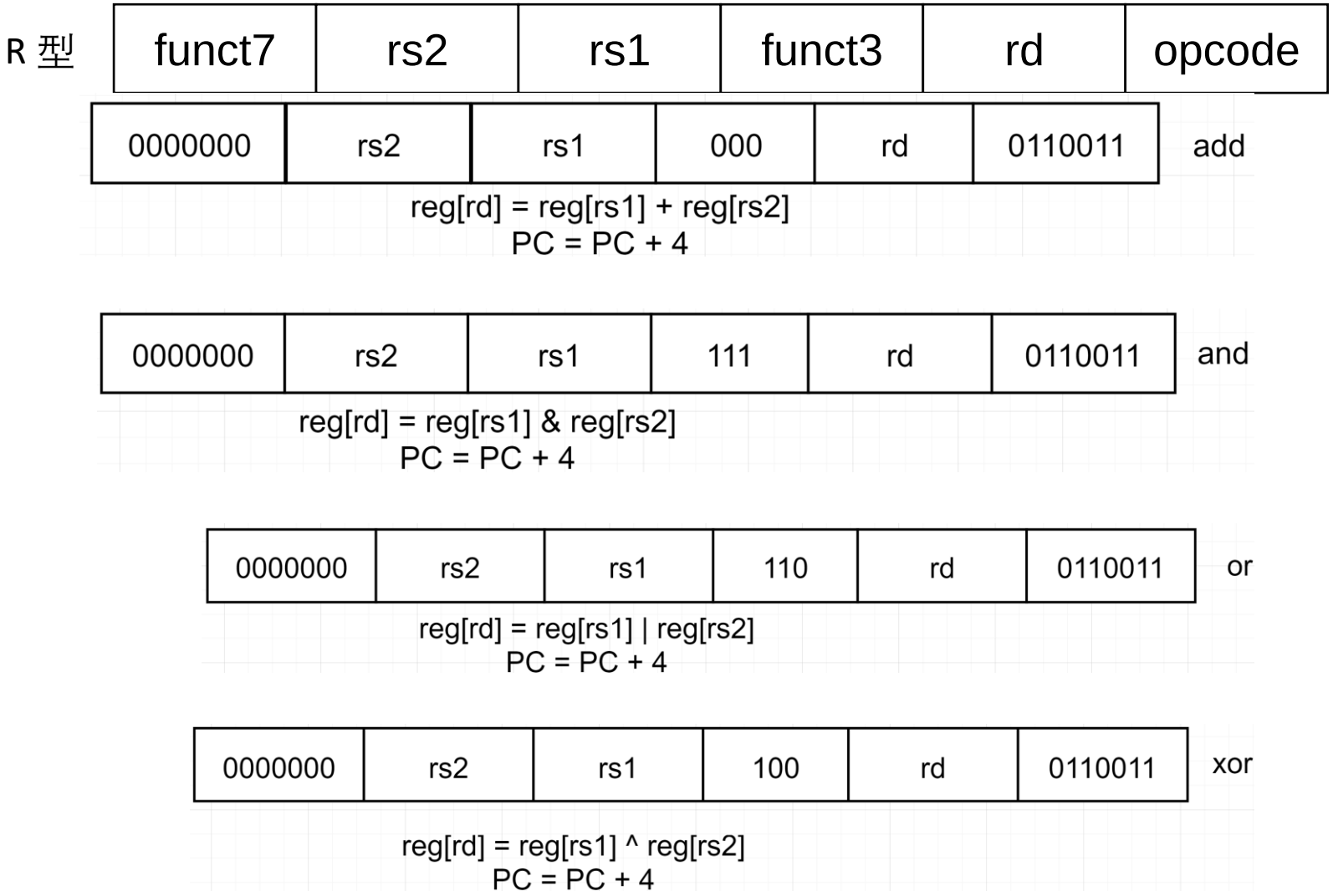


ThinPAD RV32指令系统概况

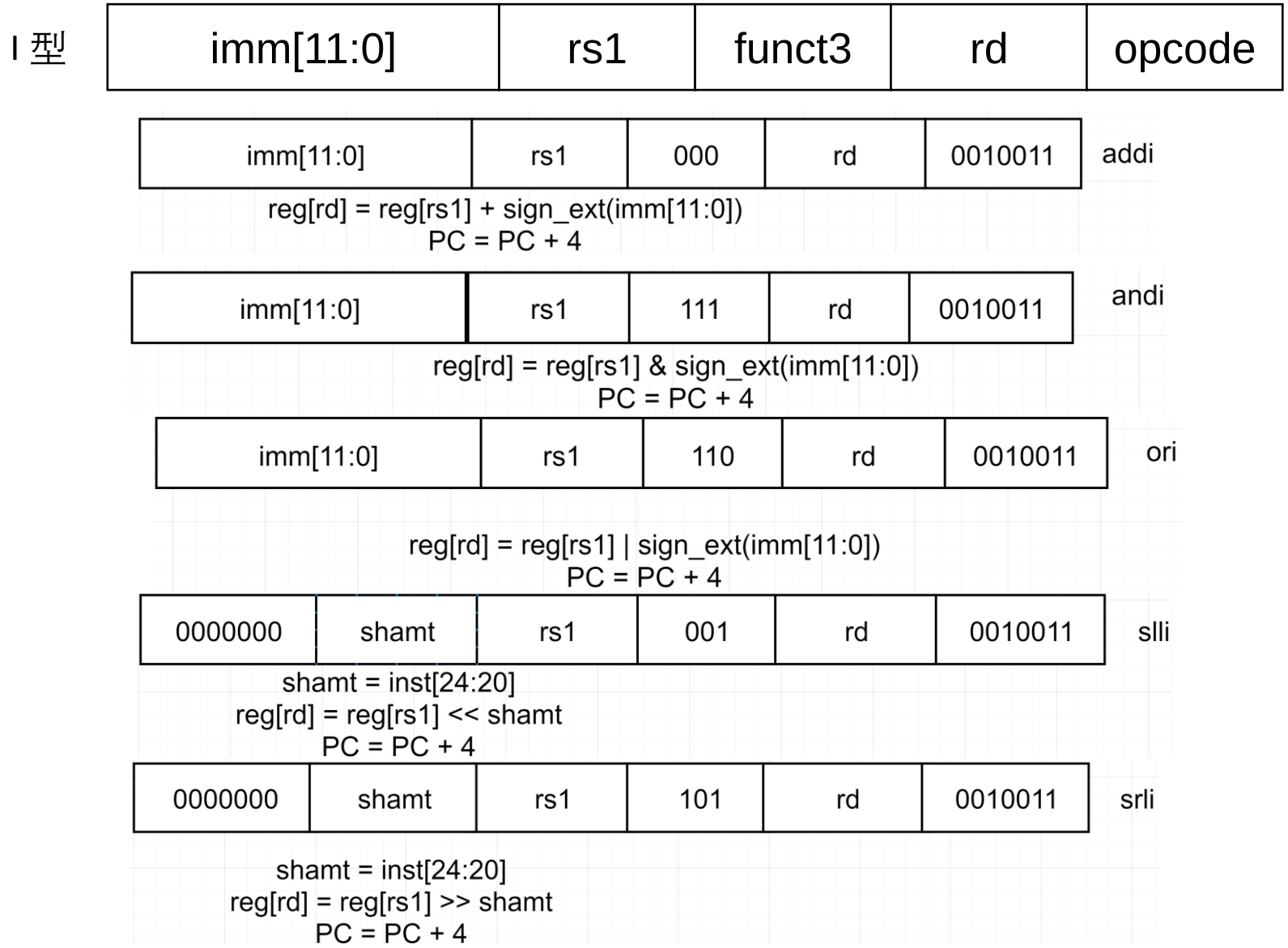
所需要实现的基础指令19条，后续只统计这19条的情况

监控程序基础版本	ADD, ADDI, AND, ANDI, AUIPC, BEQ, BNE, JAL, JALR, LB, LUI, LW, OR, ORI, SB, SLLI, SRLI, SW, XOR
监控程序支持中断版本	基础版本所有指令+CSRRC, CSRRS, CSRRW, EBREAK, ECALL, MRET
监控程序支持TLB版本	上一行所有指令+SFENCE.VMA

ThinPAD RV32指令（R型）



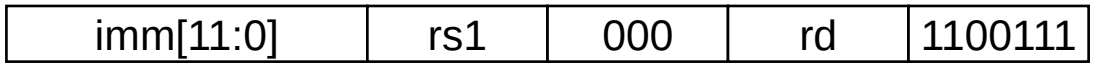
ThinPAD RV32指令（I型-1）



ThinPAD RV32指令（I型-2）

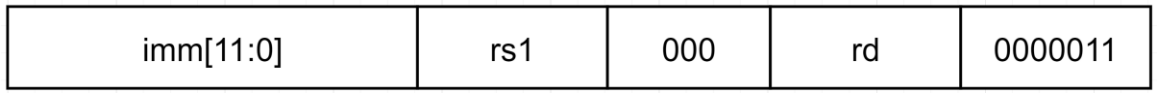
I 型





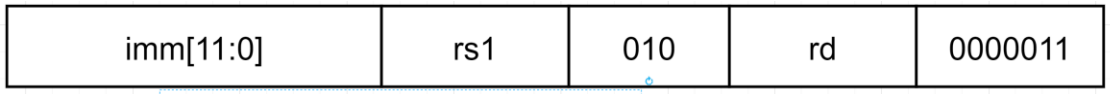
jalr

Imml = sign_ext(imm[11:0])
Reg[rd] <- pc + 4
Pc <- {(reg[rs1] + imml)[31:1], 1'b0}



lb

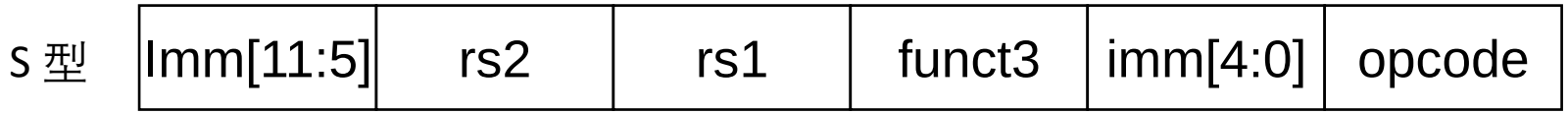
reg[rd] <= sign_ext(mem[reg[rs1] + imml] [7:0])
pc = pc + 4



lw

reg[rd] <= mem[reg[rs1] + imml]
pc = pc + 4

ThinPAD RV32指令（S/B型）



imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb

mem[reg[rs1] + sign_ext(imm[11:0]) <= reg[rs2] [7:0] byte store
pc = pc + 4

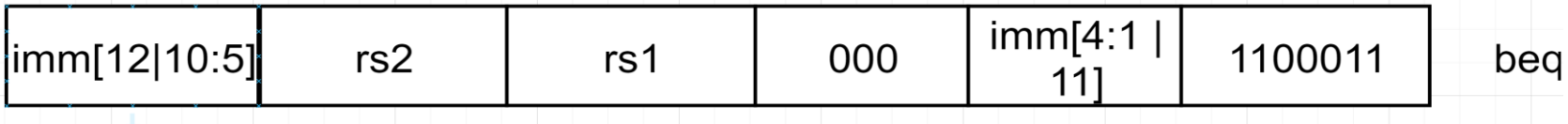
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw

mem[reg[rs1] + sign_ext(imm[11:0])] <= reg[rs2]
pc = pc + 4

ThinPAD RV32指令（S/B型）

B 型





beq

```
branch = (reg[rs1] == reg[rs2])
immB = sign_ext({imm[12:1],1'b0})
pc <= branch ? pc + immB : pc + 4
```

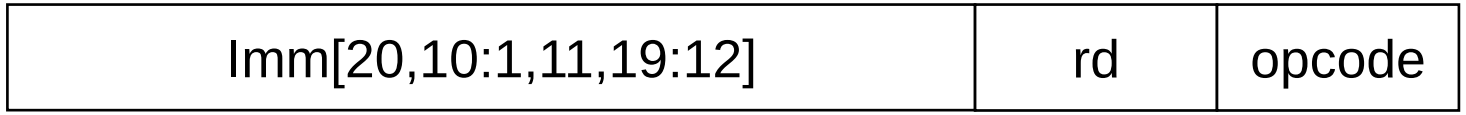


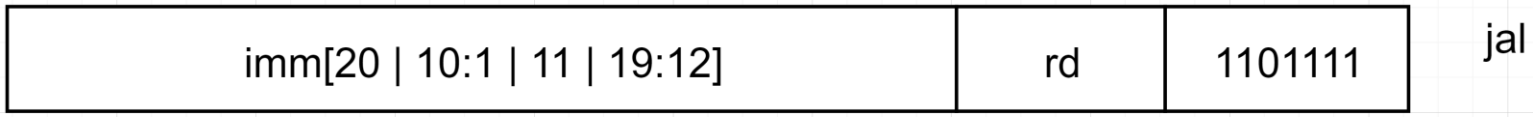
bne

```
branch = ~(reg[rs1] == reg[rs2])
immB = sign_ext({imm[12:1],1'b0})
pc <= branch ? pc + immB : pc + 4
```

ThinPAD RV32指令（U/J型）

J 型





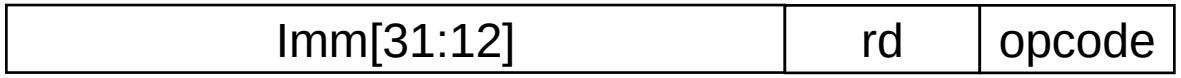
jal

immJ = sign_ext(imm[20:1], 1'b0)
reg[rd] <- pc + 4
pc <- pc+immJ

ThinPAD RV32指令（U/J型）

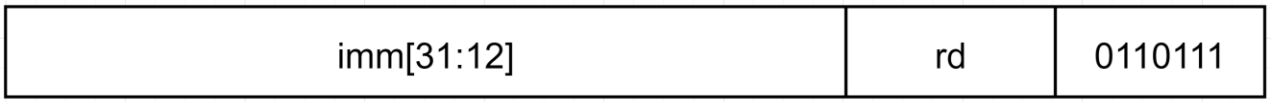
U 型





auipc

immU = inst[31:12]
Reg[rd] <= {immU, 000000000000} + pc
Pc = pc +4

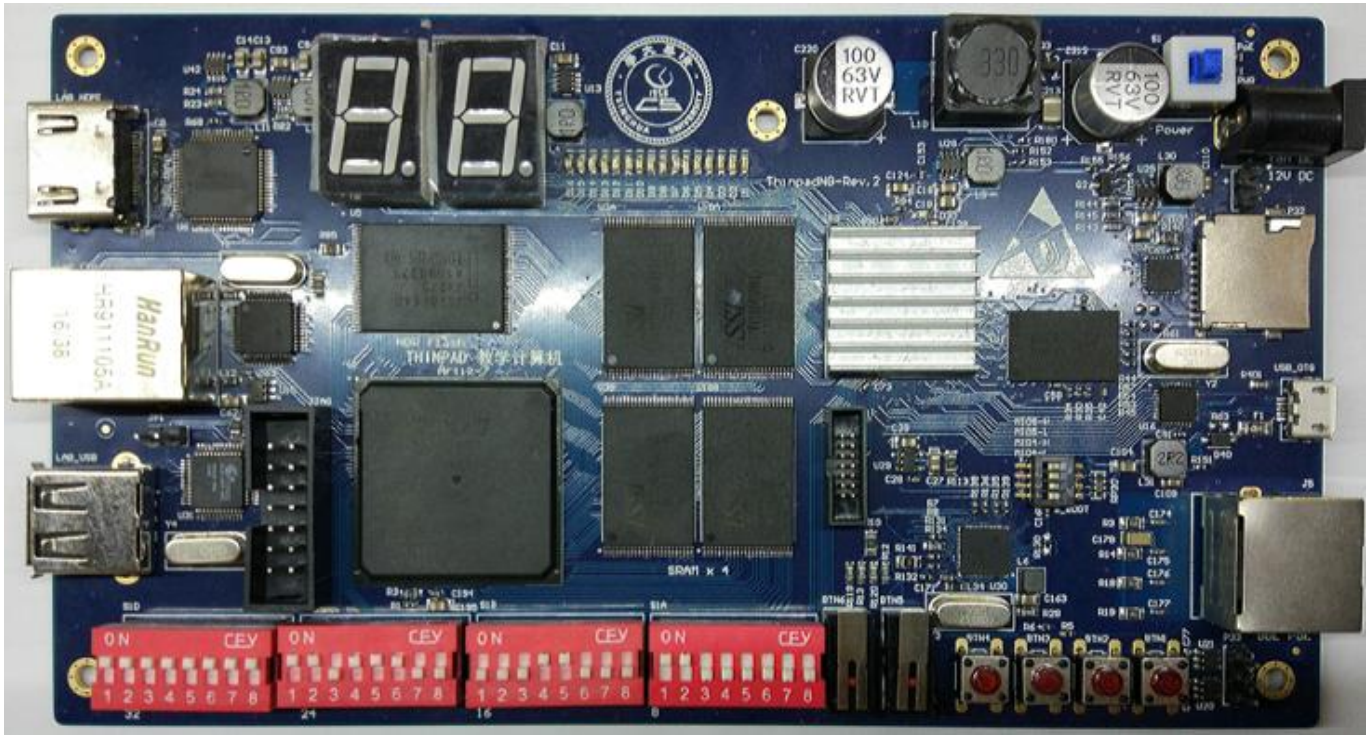


lui

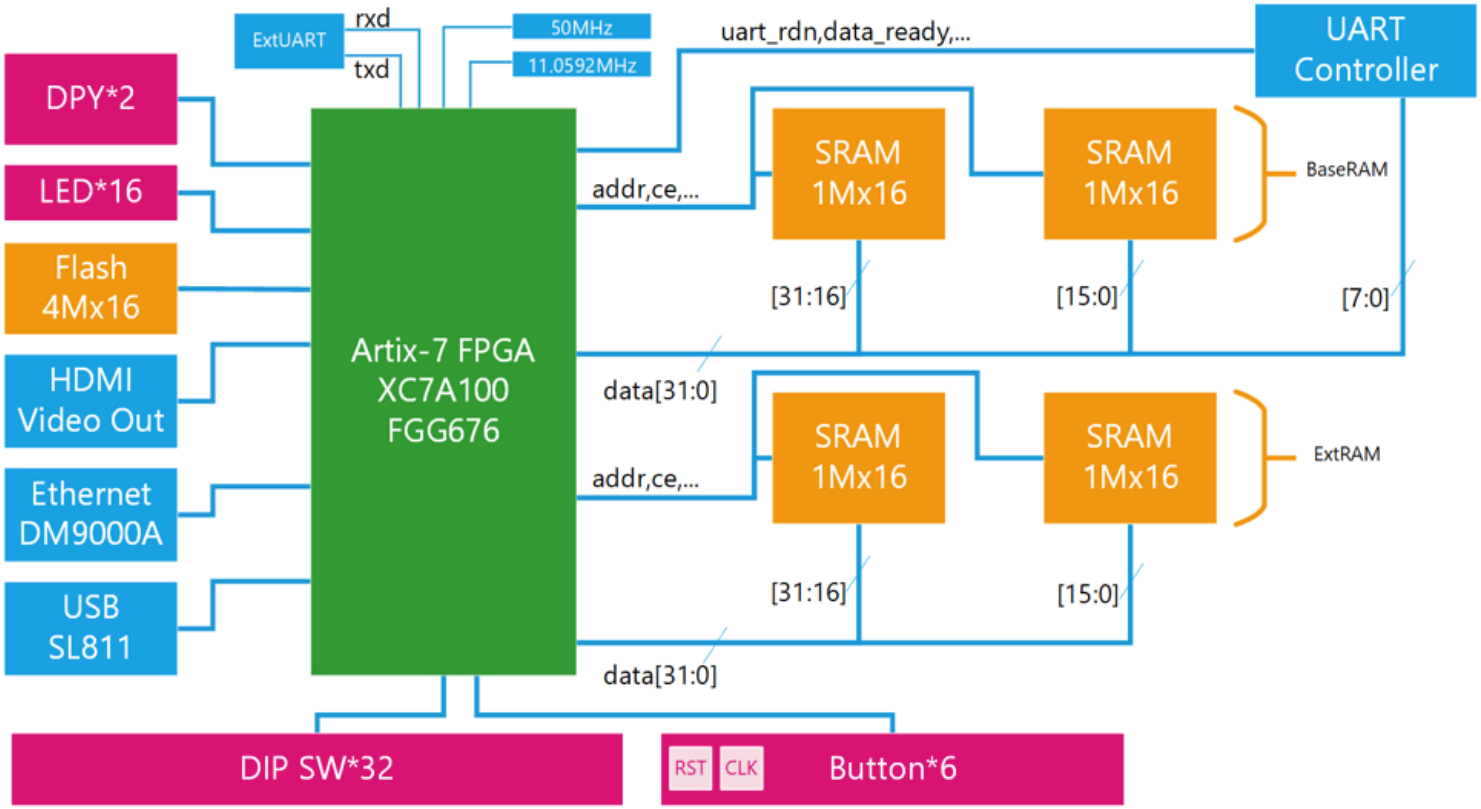
immU = inst[31:12]
reg[rd] <= {immU, 000000000000}
pc = pc + 4

ThinPAD的硬件组成

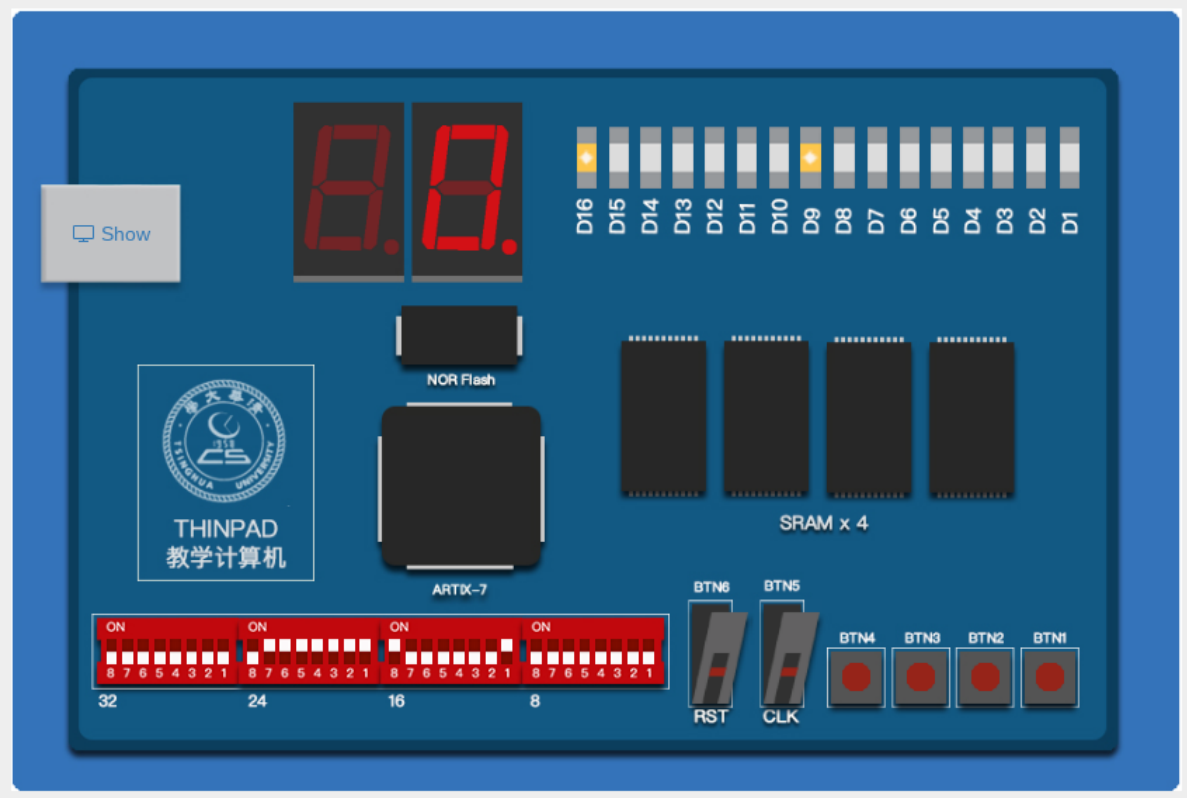
- 主要由一块FPGA，四块SRAM，串口以及其它的一些外围接口电路组成



硬件平台的内部电路构成



网络控制界面


Show
D16 D15 D14 D13 D12 D11 D10 D9 D8 D7 D6 D5 D4 D3 D2 D1
NOR Flash
SRAM x 4
THINPAD
教学计算机
ARTIX-7
ON
8 7 6 5 4 3 2 1
32 24 16 8
BTN6 BTN5 BTN4 BTN3 BTN2 BTN1
RST CLK

监控程序的地址空间划分

虚地址区间	说明
0x80000000-0x800FFFFF	Kernel代码空间
0x80100000-0x803FFFFF	用户代码空间
0x80400000-0x807EFFFF	用户数据空间
0x807F0000-0x807FFFFF	Kernel数据空间
0x10000000-0x10000008	串口数据及状态

串口寄存器位定义

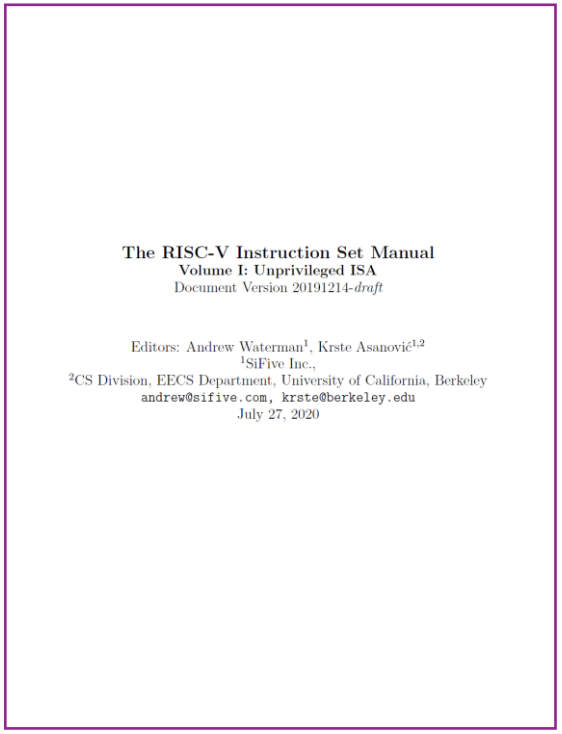
地址	位	说明
0x10000000（数据寄存器）	[7:0]	串口数据，读、写地址分别表示串口接收、发送一个字节
0x10000005（状态寄存器）	[5]	状态位，只读，为1时表示串口空闲，可发送数据
0x10000005（状态寄存器）	[0]	状态位，只读，为1时表示串口收到数据

模拟器

- 模拟器可以模拟ThinPAD硬件的执行，通过QEMU来模拟RISC-V处理器
- 监控程序可以直接运行在模拟器上
- 模拟器可以运行在Windows，Linux和Mac三个平台
- 不建议在Mac下面运行，因为后续的实验环境Vivado只能在Linux和Windows下面执行
  - 在Mac下面建议安装虚拟机，使用VirtualBox就行

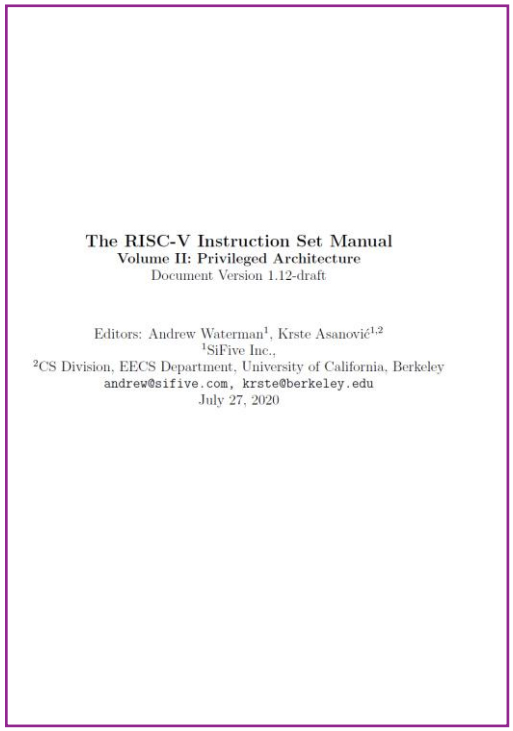
实验参考

- RISC-V指令手册，用户层编程手册，系统编程手册（版本有更新，对thinpad指令没有影响）



The RISC-V Instruction Set Manual
Volume I: Unprivileged ISA
Document Version 20191214-*draft*

Editors: Andrew Waterman[1], Krste Asanović[1,2]
[1]SiFive Inc.,
[2]CS Division, EECS Department, University of California, Berkeley
andrew@sifive.com, krste@berkeley.edu
July 27, 2020



The RISC-V Instruction Set Manual
Volume II: Privileged Architecture
Document Version 1.12-draft

Editors: Andrew Waterman[1], Krste Asanović[1,2]
[1]SiFive Inc.,
[2]CS Division, EECS Department, University of California, Berkeley
andrew@sifive.com, krste@berkeley.edu
July 27, 2020

实验提示

- 监控程序在kern下，命令行程序在term下
- 仔细阅读监控程序下的README.md，这个文件非常重要，包含了监控程序的指示，以及需要实现的指令和格式
- 按照实验指导材料运行监控程序，编写汇编代码，熟悉监控程序

实验环境说明

- 所有的实验都可以云上进行，而不需要直接使用实验板子，本年度实验不发实验板子
- 在线实验能够支持运行监控程序，ucore等，可以完成基本实验
- 后期做扩展实验的小组，有需要的提出申请，每一个小组可以借出一套实验装置，期末之前归还

小结

- 计算机程序语言
  - 高级语言
  - 汇编语言
  - 机器语言
- 指令系统
  - 硬件/软件接口
  - 指令功能/指令格式
- ThinPAD RISC-V指令系统

阅读和思考

- 阅读
- 思考
  - 指令系统的作用和地位？
  - 为实现ThinPAD指令系统，ALU应该具备哪些功能
  - 数据在计算机内部如何表示？应表示哪几类数据？
- 练习
  - 分析ThinPAD RISC-V指令系统的特点

谢谢

RISC-V指令集体系结构

- RISC-V是一个基于精简指令集（RISC）原则的开源指令集架构（ISA）。该项目2010年始于Berkeley大学，许多贡献者是该大学以外的志愿者和工业界工作者
- 其设计使其适用于各种现代计算设备（云计算机中的服务器、台式机、智能手机和嵌入式系统等）
- RISC-V 指令使用模块化的设计, 包括几个可以互相替换的基本指令集, 以及额外可以选择的扩展指令集。所有基本跟扩展的指令集都是由科技产业，研究机构跟学术界合作开发的。基本指令集规范了指令跟他们的编码，控制流程，寄存器数目（以及它们的长度），存储器跟定址方式，逻辑（整数）运算以及其他。只要有软件以及一个通用的编译器的支持，只用基本指令集就可以用来制作一个简单的通用型的电脑。
- 标准的扩展指令集可以搭配所有的基本指令集，以及其他扩展指令集，而不会冲突。